Architecture Decision Record (ADR-002)

Selección de Arquitectura de Monolito Modular y Plan de Respaldo en Capas

Jeremy Hidalgo
Santiago Ballestero
Kendall Chacón
Proyecto MiniGestor

Octubre 2026

1. Contexto

Durante una sesión de trabajo virtual realizada vía Zoom entre los integrantes del grupo de desarrollo, se analizó y definió el estilo arquitectónico del backend para el sistema **MiniGestor**.

Dado que el proyecto tiene una duración limitada de nueve semanas, se evaluó la necesidad de estructurar el código de manera que se garantice una alta cohesión, un bajo acoplamiento y una separación clara de responsabilidades sin incurrir en una complejidad excesiva que comprometa los tiempos de entrega.

2. Alternativas Evaluadas

- **Microservicios:** Descartada debido a la sobrecarga operativa, la complejidad de despliegue y la red de comunicación innecesaria para la escala y el tiempo del proyecto actual.
- **Monolito Tradicional:** Ofrece simplicidad inicial, pero tiende a entrelazar la lógica de los componentes (usuarios, ventas, inventario), dificultando el mantenimiento y la escalabilidad modular.
- **Monolito Modular (Opción Seleccionada):** Permite organizar el código en módulos independientes y bien definidos dentro de un mismo despliegue, logrando los beneficios de separación de dominios de los microservicios sin su complejidad de infraestructura.
- **Arquitectura en Capas (Opción de Respaldo / Contingencia):** Estructura clásica basada en separación horizontal (Presentación, Lógica de Negocio y Datos). Se establece como plan de contingencia en caso de que surjan limitaciones técnicas imprevistas con la modularización estricta.

3. Decisión

Se toma la decisión unánime de adoptar una **Arquitectura de Monolito Modular** para el desarrollo de MiniGestor, organizando el backend de Django en módulos independientes (usuarios, productos, inventario, ventas, proveedores, ETL y agente).

Asimismo, se acuerda formalmente que, si el equipo encuentra bloqueos técnicos significativos en la separación modular durante las primeras fases de integración, se aplicará de inmediato un esquema de **arquitectura en capas** tradicional como alternativa de respaldo segura.

4. Justificación

Las razones principales que motivaron esta decisión son:

1. **Alineación con el Tiempo del Proyecto:** El monolito modular ofrece un balance óptimo entre orden arquitectónico y velocidad de desarrollo para un ciclo de 9 semanas.
2. **Independencia de Módulos:** Facilita que cada integrante del equipo trabaje en componentes específicos (ej. ventas, inventario o ETL) reduciendo los conflictos de código en Git.
3. **Flexibilidad del Plan de Respaldo:** Contar con la arquitectura en capas como alternativa garantiza que, ante cualquier contratiempo técnico con los límites modulares de Django, el proyecto no se detendrá y mantendrá una estructura limpia y comprensible.

5. Consecuencias

- **Positivas:** Código altamente organizado, responsabilidades claras por componente, facilidad de pruebas unitarias por módulo y una curva de aprendizaje manejable para el equipo.
- **Negativas / Mitigación:** Riesgo de acoplamiento indebido entre aplicaciones de Django si no se definen bien las interfaces internas. Esto se mitiga mediante revisiones de código cruzadas y el respeto estricto a las responsabilidades documentadas en el diagrama C4 de nivel 3.